Building a Simple ETL Pipeline

Components of data pipelines · Extract, Transform, Load · Making it run again tomorrow

Moussa Doumbia, Ph.D.
Expert in DS, ML & AI
Department of Mathematics
Howard University

MATH 153 — Data Wrangling / Data Engineering

Today

1. What a pipeline is
2. Extract
3. Transform
4. Load
5. The whole thing, in one function
6. Protecting it

What a pipeline is

The job

- The Bison Café sends us **one sales file per day**.
- We have three days.
- We want a clean table, and a daily revenue report.

> And we want it to work again tomorrow, *without us*.

Definition

Definition

A **data pipeline** is a series of steps that moves data from where it *is* to where it is *useful*, doing the same thing every time it runs.

- The important words are every time it runs.
- A notebook you run once by hand is an **analysis**.
- A pipeline runs tomorrow, and next Tuesday, without you.

The five components

#	Component	What it means	In our example
1	**Source**	where the data comes from	3 daily CSVs from the till
2	**Extract**	read it in	`pd.read_csv`
3	**Transform**	make it usable	fix prices, fix names, join the menu
4	**Load**	put it where people will use it	a SQLite table + a summary CSV
5	**Schedule**	decide when it runs	every morning at 6 a.m.

> Steps 2, 3 and 4 are so common they have a name of their own: ETL.

E — T — L

```
SOURCE       EXTRACT      TRANSFORM       LOAD        SCHEDULE
3 CSVs   --> read     --> clean+join --> cafe.db <-- daily
+ menu                                   summary      6 a.m.
```

- **Extract** — get the raw data, *without changing it*.
- **Transform** — clean it, reshape it, combine it, calculate from it.
- **Load** — write the result somewhere other people can read.

ETL or ELT?

- **ETL** — transform first, then load the clean result.
- **ELT** — load the raw data first, transform it *inside* the database.

> Same three letters, different order. ELT suits very large data and powerful warehouses. ETL suits what we are doing today.

Extract

Look at the data first

```
sale_id,sale_date,sale_time,item,qty,unit_price
S-5001,2027-03-02,14:32,FRUIT CUP,2,$4.00
S-5002,2027-03-02,15:22,Turkey Wrap,2,$7.50
S-5004,2027-03-02,11:58,latte,3,$4.25
```

Before the next click

Two things in this file are already wrong. What are they?

Two problems, both normal

- `unit_price` **is text, not a number**. It says `$4.25`.
- You cannot multiply by `$4.25`.
- `item` **is spelled inconsistently** — `FRUIT CUP`, `latte`, `" Bagel "`.
- The till lets staff type it by hand.

> Fixing those two things is what “Transform” will mean today.

Extract: read, and change nothing

```python
def extract(data_dir):
    """Read every daily sales file and stack them into one DataFrame."""
    files = sorted(data_dir.glob("sales_2027-*.csv"))
    frames = [pd.read_csv(f) for f in files]
    return pd.concat(frames, ignore_index=True)
```

Output

```
107 rows from 3 files
```

> **Extract is deliberately boring.** Keeping the raw data untouched means that when something goes wrong later, you know where to start again.

What `glob` is for

```
files = ["sales_2027-03-02.csv", "sales_2027-03-03.csv", ...]  # breaks Friday
files = sorted(data_dir.glob("sales_2027-*.csv"))              # just works
```

- `glob` finds files by pattern instead of by name.
- So tomorrow's file is picked up without editing the code.
- The pattern also *excludes* `menu.csv`, which lives in the same folder.
- `sorted` matters: `glob` returns files in filesystem order, not date order.

Transform

Step 1 — make the price a number

```
df["unit_price"] = (df["unit_price"]
                      .str.replace("$", "", regex=False)
                      .astype(float))
```

Output

```
before: $4.00    (a str)
after : 4.0      (a float64)
```

> `regex=False` because `$` means “end of string” in a regular expression. Here we want the literal dollar sign.

Step 2 — make the date a real date

```
df["sale_date"] = pd.to_datetime(df["sale_date"], format="%Y-%m-%d")
```

- A date stored as text is just a string.
- You cannot sort it reliably, subtract it, or ask what day of the week it was.
- Convert it *once*, here, and every later step gets a real date.

Step 3 — clean the item names

```
df["item"] = df["item"].str.strip().str.title()
```

Output

```
distinct spellings before: 29
distinct item names after:  8
```

- `.str.strip()` removes spaces at the ends — `" Latte "` becomes `"Latte"`.
- `.str.title()` capitalises each word — `"LATTE"` and `"latte"` both become `"Latte"`.

Step 4 — join the menu, then calculate

```
df = df.merge(menu, on="item", how="left")
df["revenue"] = (df["qty"] * df["unit_price"]).round(2)
```

Before the next click

Why did step 3 have to come *before* step 4?

Watch out

`"latte"` would not have matched `"Latte"` in the menu. That sale would have had no category, and its revenue would have quietly vanished from the report.

Two checks after every join

- **Did the row count stay the same?** 107 in, 107 out.
- If it grew, the reference table has duplicate rows and every sale got counted twice.
- **Did anything fail to match?** Zero unmatched here.
- A blank category means a spelling your cleaning step missed.

In practice

Print both numbers. A join that silently doubled your data looks exactly like one that worked.

Load

Two outputs, two audiences

- A **database table** — for anyone who wants to query it.
- A **summary CSV** — for the manager who just wants the daily numbers.

> “Load” does not mean one place. It means wherever the people who need the answer will actually look.

Load

```
con = sqlite3.connect("cafe.db")
out.to_sql("sales", con, if_exists="replace", index=False)
con.close()

summary.to_csv("daily_summary.csv", index=False)
```

> Dates go into SQLite as `'YYYY-MM-DD'` text, because SQLite has no date type — and in that format, alphabetical order *is* chronological order.

The result

sale_date	category	items_sold	revenue
2027-03-02	Drink	35	$119.75
2027-03-02	Food	40	$222.85
2027-03-03	Drink	48	$162.80
2027-03-03	Food	29	$112.40
2027-03-04	Drink	39	$124.30
2027-03-04	Food	29	$126.25
Total			**$868.35**

> 107 sales over three days, from two messy columns and a join.

The whole thing, in one function

run_pipeline()

```python
def run_pipeline(data_dir=DATA, db_path="cafe.db", csv_path="daily_summary.csv"):
    """Extract -> Transform -> Load. Run this every morning."""
    menu  = pd.read_csv(data_dir / "menu.csv")

    raw   = extract(data_dir)                          # E
    clean = transform(raw, menu)                       # T

    assert clean["category"].notna().all(), "some items did not match the menu"
    assert (clean["revenue"] > 0).all(),    "a sale has zero or negative revenue"

    summary = (clean
               .assign(day=clean["sale_date"].dt.date)
               .groupby(["day", "category"], as_index=False)
               .agg(items_sold=("qty", "sum"), revenue=("revenue", "sum"))
               .rename(columns={"day": "sale_date"}))

    load(clean, summary, db_path, csv_path)            # L
    return clean, summary
```

> Twelve lines. This is the deliverable — not the notebook, the function.

Why a function, and not just cells?

- **It has a name.**
- You can say “run `run_pipeline()`” instead of “run cells 4 to 19 but not cell 12”.
- **It takes arguments.**
- Point it at a different folder and it works on different data.
- **It can be scheduled.**
- A computer can call a function at 6 a.m. It cannot click through a notebook.

Protecting it

Run it twice

```
run_pipeline()
run_pipeline()          # on purpose
```

Before the next click

How many rows are in the table now?

```
rows in the sales table: 107        <-- not 214
```

> Running it twice gave the same answer as running it once. That property has a name: the pipeline is idempotent.

One keyword makes the difference

```
out.to_sql("sales", con, if_exists="replace", index=False)    # yes
out.to_sql("sales", con, if_exists="append",  index=False)    # NO
```

Watch out

With `append`, every run adds another copy. Run it three times and the café has sold everything three times.

This is the single most common bug in a first pipeline.

A log line per step

```
EXTRACT    107 rows read
TRANSFORM  107 rows cleaned, 8 distinct items
LOAD       107 rows written, revenue $868.35
```

- Three `print` statements.
- Now when the job runs at 6 a.m. and something looks odd . . .
- . . . you can see which step the rows disappeared at, instead of guessing.

A check before you publish

```
assert clean["category"].notna().all(), "some items did not match the menu"
assert (clean["revenue"] > 0).all(),    "a sale has zero or negative revenue"
```

- An **assertion** is a sentence that must be true.
- If it is not, Python stops immediately and tells you why.

Before the next click

Where in the pipeline should these two lines go?

Between Transform and Load

- That is the last moment before the numbers reach somebody else.
- If a check fails, the pipeline stops *before* writing to the database.
- The manager's report is still yesterday's.
- Which is correct — and much better than today's report being wrong.

In practice

A pipeline that publishes a wrong number is worse than a pipeline that publishes nothing.

Watch a check fail

```
broken = clean.copy()
broken.loc[0, "revenue"] = -5.0          # pretend the transform has a bug

assert (broken["revenue"] > 0).all(), "a sale has zero or negative revenue"
```

Output

```
Pipeline stopped: a sale has zero or negative revenue
```

> Nothing was written to the database. The bad number never left the building.

The same summary, in SQL

```
SELECT sale_date, category,
       SUM(qty)               AS items_sold,
       ROUND(SUM(revenue), 2) AS revenue
FROM   sales
GROUP  BY sale_date, category
ORDER  BY sale_date, category;
```

> Once the data is in the database, the report is one query. Same answer, different language.

pandas or SQL?

Use pandas when	Use SQL when
reading files and cleaning text	the data is already in a database
the step is awkward to express in SQL	the data is too big for memory
you are still exploring	the question is a standard group-and-total

> Most real pipelines use both: pandas to clean and load, SQL to query afterwards. That is exactly what we did.

Five things to remember

1. **Extract changes nothing.** Keep the raw data raw, so you can always start over.
2. **Clean before you join.** `"latte"` does not match `"Latte"`.
3. **Check the row count after every join.** Same number in, same number out.
4. **Make it runnable twice.** `if_exists="replace"`, not `"append"`.
5. **Put the checks between Transform and Load.**

The one-sentence version

A pipeline is just three functions and a schedule.

The difference between a pipeline and a notebook
is that somebody else can run it.

In-class activity

Founders Library checkouts

Three daily files. `loan_days` arrives as `"14 days"`. `patron_type` is typed inconsistently. Build the pipeline.

- Six tasks, about 35 minutes, in pairs.
- Same five components — only the data is different.
- Notebook: `ETL_Activity.ipynb`

In practice

Expected: **82 rows**, **16 spellings become 4**, and the table still says 82 after you run it twice.